Big-O in Practice

Linear Scan vs. Dict Lookup With a Partner

```python
names = ["Gum", "Tape"]
print("Tape" in names)
# O(n) above, O(1) below
stock = {"Tape": 30}
print(stock["Tape"])
```

By the End of Today, You'll Be Able To…

1

Write both search approaches over real objects

2

Determine and defend each one's Big-O

You'll Know These by the End of Today

Linear-scan search

Check items one at a time until a match appears.

Dict-keyed lookup

Jump directly to a value using its key.

Reading Recap: Straight to the Slot

Today's Reading: Hash Maps

A hospital front desk types a record number. The chart must appear at once, with 500 patients or 5 million. Reopen the reading: Straight to the Slot: Hash Maps

The Scenario: A Hospital Front Desk

500 patients: checking chart by chart for MRN-77190 is barely noticeable.

5 million patients: the same chart-by-chart scan takes real, felt time.

A hash map turns MRN-77190 into 26. `26 % 8` is 2: straight to slot 2.

READING RECAP

Key Terms and Code From the Reading

HASH FUNCTION
Turns a key into a number, the same number every time.

SLOT (BUCKET)
`26 % 8` reduces the number to one of 8 slots: slot 2.

COLLISION, CHAINING
Two keys land in one slot. The slot keeps a small list, so both stay.

TWO SUM
`complement in seen` checks a dict each step: one pass, not a pair-by-pair scan.

READING CHECK

MRN-19004 hashes to 59. The table has 8 slots. Which slot does it land in?

A

Slot 3

B

Slot 7

C

Slot 59

ANSWER KEY

Answer Key: From Hash Code to Slot

Correct answer: A. Slot 3

`59 % 8` is 3: 8 times 7 is 56, with 3 left over. A table with 8 slots only has indexes 0 to 7.

READING CHECK

In brute-force Two Sum, the list of amounts doubles. How many pair comparisons now?

A

About the same

B

About 2 times as many

C

About 4 times as many

ANSWER KEY

Answer Key: Nested Loops Grow Fast

Correct answer: C. About 4 times as many

Nested loops compare every amount with every other one. Double the amounts and the comparisons roughly quadruple: O(n²).

Your Questions From the Reading

Turn and talk for 1 min. Bring one question.

We take 2 or 3 questions now.

Bigger ones wait for today's matching slide.

IN THE NEWS

One Player Found the Slow Search. Load Time Fell 70%.



The game loaded about 63,000 items
Each new item rescanned the whole list
Rockstar patched it, paid the player $10,000

The game checked each new item against its whole list, one by one. What would you have used instead?

Source: t0st, nee.lv, Feb 28, 2021; Rockstar statement via VGC

PARTNER UP

Two Skills, Together



Big-O intuition and class syntax have been separate lessons. Today combines them with a partner.

Quick Recap: Linear Scan vs. Dict Lookup

Linear scan

One item at a time

Dict lookup

Jump straight to the match

Today

Both approaches, on real objects

CODE ALONG

Open the Code-Along

Open this file:
`Module1_Lesson15_big-o-in-practice-linear-scan-vs-dict-lookup-with-a-partner_STUDENT.py`
Run it after every section: `python3` + the file name

Sections, in order:

1. Setup: A Class and Its Objects
   A real class and a real list of its objects come first, before either search function.
2. Search 1: Linear Scan
   Write the search the same way you'd check a list by hand: one item at a time.
3. Search 2: Dict-Keyed Lookup
   Same data, same question: a completely different mechanism.

Setup: A Class and Its Objects

python

```python
class SupplyItem:
    def __init__(self, name, count, department):
        self.name = name
        self.count = count
        self.department = department
supplies = [
    SupplyItem("Stapler", 12, "Office"),
    SupplyItem("Paper", 200, "Office"),
    SupplyItem("Gloves", 50, "Warehouse"),
    SupplyItem("Tape", 30, "Office"),
]
```

Search 1: Linear Scan

python

```python
def find_by_name_linear(supplies, name):
    for item in supplies:
        if item.name == name:
            return item
    return None

found = find_by_name_linear(supplies, "Tape")
print(found.name, found.count)
```

Search 2: Dict-Keyed Lookup

python

```
supplies_by_name = {item.name: item for item in
supplies}

found = supplies_by_name.get("Tape")
print(found.name, found.count)
```

CODE-ALONG DONE

Back to the Slides: Review

You just typed it and ran it. Now check that it stuck, without the code in front of you.

QUICK CHALLENGE

Predict it: What does `find_by_name_linear(supplies, "Nope")` return?

A The last item in the list

B An error

C `None`

ANSWER KEY

Linear Scan, No Match

Correct answer: None

The loop finishes without matching, so it falls through to `return None`. Worst case: it checks every item.

QUICK CHALLENGE

Predict it: What does `supplies_by_name.get("Nope")` return?

A `None`

B An error

C An empty `SupplyItem`

ANSWER KEY

Dict Lookup, No Match

Correct answer: None

`.get()` returns `None` by default when the key isn't found. No scan happened; the dict simply lacked that key.

ADD TO THE BOARD

Post Each Search's Big-O

Linear scan

Post its Big-O and explain why.

Dict lookup

Post its Big-O and explain why it scales better.

Watch It Run: Scan vs. Lookup

Step 1 of 7

Linear scan: look for "Tape" in a list of 4 items

```
def find_by_name_linear(supplies, name):
    for item in supplies:
        if item.name == name:
            return item
found = find_by_name_linear(supplies, "Tape")
supplies_by_name = {item.name: item for item in supplies}
found = supplies_by_name.get("Tape")
print(found.name, found.count)
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

Same Result. Very Different Search Cost.

`find_by_name_linear`

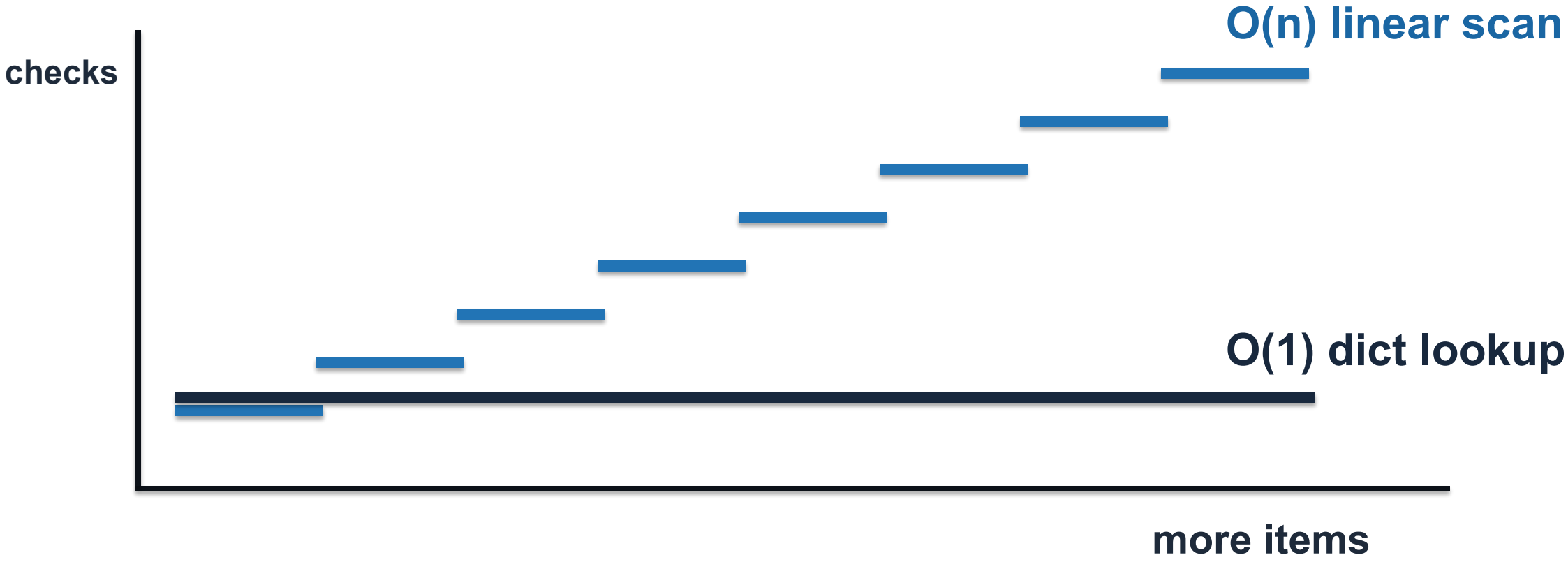
O(n)
One item at a time
Cost grows with list size

`supplies_by_name.get()`

O(1)
Jumps straight to the key
Cost stays flat

Both return the same object — or `None`.

How the Search Cost Grows



QUICK CHALLENGE

If `supplies` grows to 10,000 items, what's the worst-case number of checks for `find_by_name_linear`?

A

1

B

10,000

C

5,000

ANSWER KEY

Scaling the Numbers

Correct answer: 10,000

Worst case, a linear scan checks every item. 10,000 items means up to 10,000 checks — no shortcuts.

DISCUSS

Would You Rather?

5 items

Linear scan or dict lookup?
Defend your choice.

5 million items

Linear scan or dict lookup?
Defend your choice.

Why This Matters for Your Project

`find_item`

Needs a real search choice

Why This Matters for Your Project

`find_item`

Needs a real search choice

Choose

Linear scan OR dict — not both, not neither

Why This Matters for Your Project

`find_item`

Needs a real search choice

Choose

Linear scan OR dict — not both, not neither

Today

Your first rep writing both

TIP

“When Would You Keep the Linear Scan?”

Answer it like an interview, with the Tradeoff Lens:

option, benefit, cost, when you'd choose it.

MODEL ANSWER

Model Answer: Tradeoff Lens

OPTION

Keep `find_by_name_linear` and scan the list I already have.

BENEFIT

No dict to build or keep in sync as items change.

COST

O(n): every search slows down as the list grows.

WHEN I'D CHOOSE IT

A tiny list or a one-time search; repeated lookups get a dict.

QUICK CHALLENGE

A Fellow says a dict lookup is always faster in real seconds than a linear scan. What's missing?

A Nothing — always true

B On a tiny list, the difference may not be measurable

C Dict lookups are always slower

ANSWER KEY

Big-O Isn’t a Stopwatch

Correct answer: On a tiny list, the difference may not be measurable

Big-O describes how cost scales as data grows — not a guaranteed real-seconds ranking at every size.

Key Terms — Recap

Linear scan

O(n)

Dict lookup

O(1)

Worst case

Maximum work

Scaling

How cost changes as data grows

So Far, We've Covered…

GTA Online

Its slow check needed a dict

`find_item`

This is exactly what it needs

Next

Same comparison, your own class

Next Class

Next class

Stress-testing your own class